Multi-Vendor Procurement & Purchase Order Management System

Final Project Report

Course: Database Management Tools

Team 3

Nisarg Shah
Smit Patel
Parva Shah
Swet Patel
Priya

1. Project Overview and Objective

Description of the project: This project is a comprehensive, full-stack enterprise procurement platform designed to simulate a real-world purchasing workflow. It digitally manages the entire lifecycle of how an organization purchases goods from external vendors, integrating role-based access control, real-time budget tracking, and extensive audit logging.

In modern corporate environments, procurement is a multi-stage process involving numerous departments. An employee must request an item, a manager must review and approve it based on budget availability, a procurement officer must issue a formal Purchase Order to a vendor, the vendor must ship the goods, and finally, the finance department must process the invoice and release payment. Our system perfectly encapsulates this entire pipeline in a centralized, web-based application.

Overall goal of the database system: The primary goal is to digitize and streamline the multi-step, multi-department procurement process. By replacing manual paperwork with a

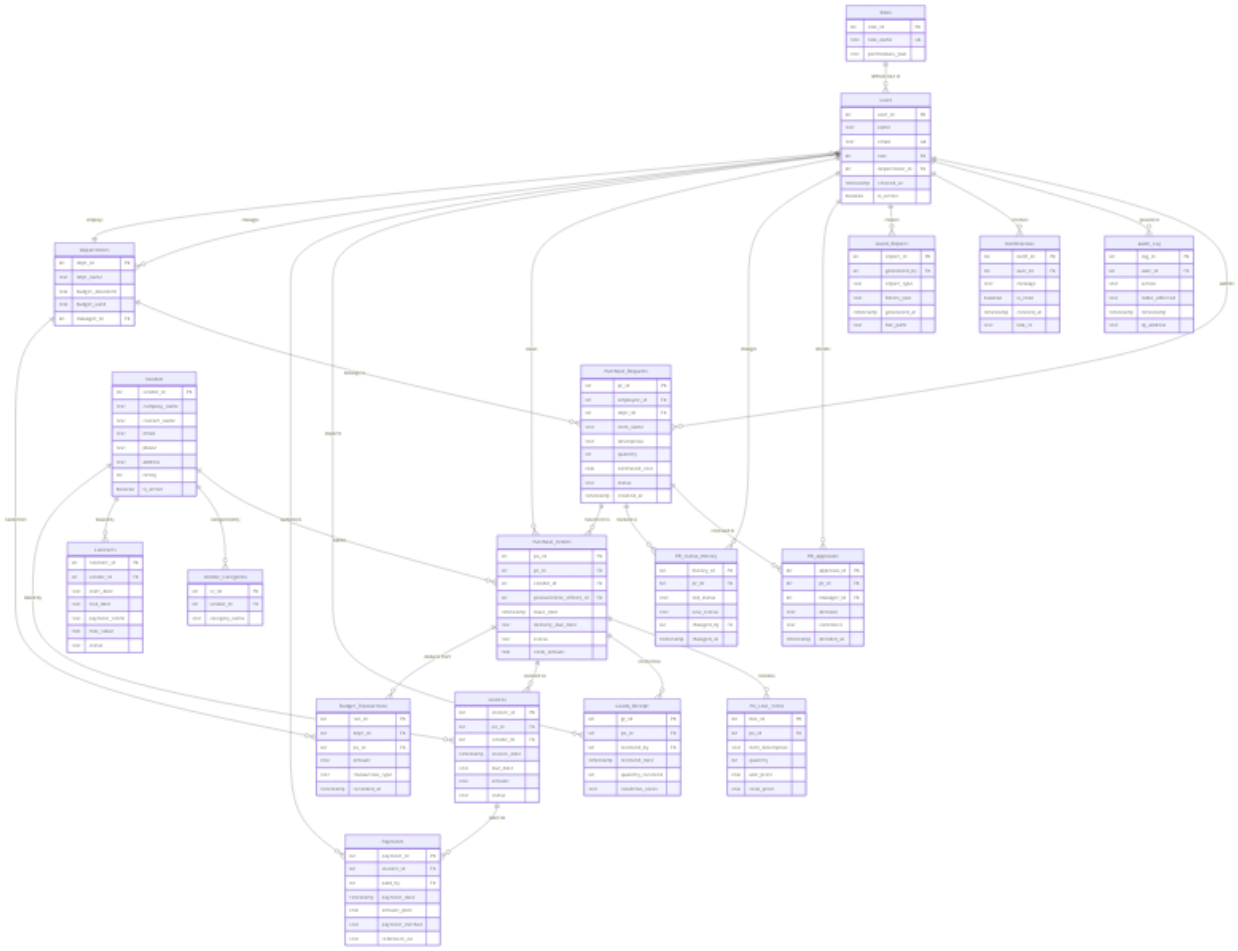
centralized relational database, the system ensures data integrity, enforces strict department budget constraints, maintains a consistent state machine for all purchase requests, and provides actionable KPI metrics for management. It prevents unauthorized spending, provides a strict audit trail, and increases overall operational efficiency.

2. Database Design

Business rules summary: The system enforces several critical business rules:

- Managers can only approve requests if the department has sufficient remaining budget. If the estimated cost exceeds the available funds, the system rejects the transaction.
- Users are strictly segregated by roles (Employee, Manager, Procurement, Vendor, Finance, Admin), and specific API endpoints and views are restricted accordingly.
- A Purchase Order cannot be generated without an approved Purchase Request.
- An Invoice cannot be processed until the associated goods have been formally received (Goods Receipt).
- Every status change in the procurement lifecycle is immutably logged in a dedicated history table.

ERD and schema explanation: The database follows a normalized relational design satisfying Third Normal Form (3NF). It consists of 16 interconnected tables divided into six functional groups: Authentication, Procurement Workflow, Vendor Management, Order Fulfillment, Finance, and System Monitoring. Below is the complete Entity-Relationship Diagram (ERD) mapping the foreign key relations across the system.



Key design decisions: Data redundancy is minimized by strictly defining foreign key relationships. Roles, Departments, and Vendor Categories are separated into distinct tables. Rather than merely overwriting a status column when an order progresses, a dedicated `PR_Status_History` table is used to create a robust audit trail of state changes.

3. Data Dictionary Overview

Description of tables, relationships, key attributes, and constraints: The database relies heavily on AUTOINCREMENT integers for Primary Keys, and NOT NULL/DEFAULT constraints to maintain integrity.

Authentication & Identity Module

1. Roles

Defines the permission levels in the system. **Columns:** `role_id` (PK), `role_name` (UNIQUE), `permissions_json`.

2. Departments

Tracks budget allocations and current expenditure per internal division. **Columns:** `dept_id` (PK), `dept_name` (NOT NULL), `budget_allocated` (REAL), `budget_used` (REAL), `manager_id` (FK to Users).

3. Users

Stores all employee and vendor credentials. **Columns:** `user_id` (PK), `name`, `email` (UNIQUE), `role` (FK to Roles), `department_id` (FK to Departments), `created_at`, `is_active`.

Procurement Workflow Module

4. Purchase_Requests

Stores item requests generated by employees. **Columns:** `pr_id` (PK), `employee_id` (FK), `dept_id` (FK), `item_name`, `quantity`, `estimated_cost`, `status`, `created_at`.

5. PR_Approvals

Records the approval or rejection decisions made by managers. **Columns:** `approval_id` (PK), `pr_id` (FK), `manager_id` (FK), `decision`, `comments`, `decided_at`.

6. PR_Status_History

Maintains the state machine transitions for requests. **Columns:** `history_id` (PK), `pr_id` (FK), `old_status`, `new_status`, `changed_by` (FK), `changed_at`.

Vendor Management Module

7. Vendors

Stores details for supplier companies. **Columns:** `vendor_id` (PK), `company_name`, `contact_name`, `email`, `phone`, `address`, `rating`.

8. Vendor_Categories

Categorizes vendors by specialty. **Columns:** `vc_id` (PK), `vendor_id` (FK), `category_name`.

9. Contracts

Records long-term vendor agreements. **Columns:** `contract_id` (PK), `vendor_id` (FK), `start_date`, `end_date`, `max_value`, `status`.

Order Fulfillment Module

10. Purchase_Orders

The official purchase document sent to vendors. **Columns:** `po_id` (PK), `pr_id` (FK), `vendor_id` (FK), `procurement_officer_id` (FK), `issue_date`, `delivery_due_date`, `status`, `total_amount`.

11. PO_Line_Items

Individual items contained within a Purchase Order. **Columns:** `line_id` (PK), `po_id` (FK), `item_description`, `quantity`, `unit_price`, `total_price`.

12. Goods_Receipt

Records when physical goods are delivered and inspected. **Columns:** `gr_id` (PK), `po_id` (FK), `received_by` (FK), `received_date`, `quantity_received`, `condition_notes`.

Finance & Payments Module

13. Invoices

Billing documents submitted by vendors. **Columns:** `invoice_id` (PK), `po_id` (FK), `vendor_id` (FK), `invoice_date`, `due_date`, `amount`, `status`.

14. Payments

Financial transactions clearing invoices. **Columns:** `payment_id` (PK), `invoice_id` (FK), `paid_by` (FK), `payment_date`, `amount_paid`, `payment_method`, `reference_no`.

15. Budget_Transactions

Ledger of department budget movements. **Columns:** `txn_id` (PK), `dept_id` (FK), `po_id` (FK), `amount`, `transaction_type`, `recorded_at`.

System & Monitoring Module

16. Audit_Log

Tracks every action with IP address and timestamp. **Columns:** `log_id` (PK), `user_id` (FK), `action`, `table_affected`, `timestamp`, `ip_address`.

4. SQL Implementation & Proof of Work

Overview of table creation scripts: The database was implemented using SQLite3. DDL scripts establish strict referential integrity. Below are the actual DDL statements used to construct the core tables.

Proof of Work: Core Table Creation

```
CREATE TABLE IF NOT EXISTS Roles (
    role_id INTEGER PRIMARY KEY AUTOINCREMENT,
    role_name TEXT UNIQUE NOT NULL,
    permissions_json TEXT
);

CREATE TABLE IF NOT EXISTS Departments (
    dept_id INTEGER PRIMARY KEY AUTOINCREMENT,
    dept_name TEXT NOT NULL,
    budget_allocated REAL DEFAULT 0,
    budget_used REAL DEFAULT 0,
    manager_id INTEGER,
    FOREIGN KEY(manager_id) REFERENCES Users(user_id)
);

CREATE TABLE IF NOT EXISTS Users (
    user_id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    email TEXT UNIQUE NOT NULL,
```

```
    role INTEGER,
    department_id INTEGER,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    is_active BOOLEAN DEFAULT 1,
    FOREIGN KEY(role) REFERENCES Roles(role_id),
    FOREIGN KEY(department_id) REFERENCES Departments(dept_id)
);

CREATE TABLE IF NOT EXISTS Purchase_Requests (
    pr_id INTEGER PRIMARY KEY AUTOINCREMENT,
    employee_id INTEGER,
    dept_id INTEGER,
    item_name TEXT NOT NULL,
    description TEXT,
    quantity INTEGER NOT NULL,
    estimated_cost REAL,
    status TEXT DEFAULT 'PENDING',
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY(employee_id) REFERENCES Users(user_id),
    FOREIGN KEY(dept_id) REFERENCES Departments(dept_id)
);
```

Description of Query Categories

The system relies on three distinct categories of SQL queries to drive the application logic:

- **Basic Queries:** Standard INSERT, SELECT, and UPDATE commands used to manage user sessions, load profile information, and generate initial dashboard views.

• **Intermediate Queries:** Multi-table JOINs used to compile comprehensive data sets, such as linking Purchase Orders to their respective Vendors and original Purchase Requests to display detailed tabular data on the front end.

• **Advanced Queries:** Complex transactional sequences that cascade status updates backwards across the workflow state machine, and aggregate functions (SUM, COUNT) used to dynamically calculate remaining department budgets and KPI metrics.

Proof of Work: Advanced Join and KPI Aggregation

This query fetches detailed Purchase Order information while simultaneously resolving the Vendor name and calculating active pipeline metrics for the dashboard.

```
SELECT
    po.po_id,
    po.issue_date,
    po.status,
    po.total_amount,
    v.company_name
FROM Purchase_Orders po
JOIN Vendors v ON po.vendor_id = v.vendor_id
WHERE po.status != "PAID";
```

Proof of Work: Advanced Transactional State Cascade

When Finance marks an invoice as PAID, the system executes a series of synchronized queries. It updates the Invoice, cascades the PAID status backwards to the Purchase Order and the original Purchase Request, logs the history, and records the Payment, all in one transaction.

```
-- Step 1: Update Invoice Status
UPDATE Invoices SET status = "PAID" WHERE invoice_id = ?;

-- Step 2: Cascade status backwards to PO and PR
UPDATE Purchase_Orders SET status = "PAID" WHERE po_id = ?;
UPDATE Purchase_Requests SET status = "PAID" WHERE pr_id = ?;
```

```
-- Step 3: Insert State History Audit Trail

INSERT INTO PR_Status_History (pr_id, old_status, new_status,
changed_by)

VALUES (?, 'INVOICED', 'PAID', ?);


-- Step 4: Record the formal Payment transaction

INSERT INTO Payments (invoice_id, paid_by, amount_paid,
payment_method, reference_no)

VALUES (?, ?, ?, ?, ?);
```

5. Frontend Integration (Optional Extra Credit)

Frontend tool/framework: The frontend is built using HTML5, Custom CSS3, and Bootstrap 5.3. We implemented a modern dark theme featuring glassmorphism (translucent backgrounds with blur effects) and dynamic blob animations. Data visualization is powered by Chart.js, rendering interactive doughnut and pie charts. The application uses the Python Flask framework with Jinja2 for server-side template rendering.

Database connection: The Flask backend connects to the SQLite database using Python's built-in sqlite3 library. A custom `get_db_connection()` function establishes the connection and configures the `sqlite3.Row` row factory. This returns database records as dictionary-like objects, which are then passed seamlessly into Jinja2 templates for dynamic HTML rendering.

Proof of Work: Database to Jinja Routing

The following Python snippet demonstrates how data is extracted from the database and routed to the frontend Jinja template:

```
@app.route('/dashboard')

@login_required

def dashboard():

    conn = get_db_connection()

    data = {}
```

```
    if current_user.role_name == 'Employee':
        data['requests'] = conn.execute(
            'SELECT * FROM Purchase_Requests WHERE employee_id = ?',
            (current_user.id,)
        ).fetchall()
        data['kpi_total'] = len(data['requests'])
        data['kpi_pending'] = len([r for r in data['requests'] if
r['status'] == 'PENDING'])


    conn.close()
    return render_template('dashboard.html', data=data)
```

Live Deployment Proof: The entire application has been deployed live on Vercel utilizing their serverless Python hosting environment. GitHub CI/CD is configured to deploy automatically on every push to the master branch. Authentication is securely managed by the Firebase Auth SDK, preventing unauthorized access.

6. Challenges and Solutions

Challenge 1: Enforcing Strict Budget Constraints

Issue encountered: We needed a foolproof way to prevent managers from approving purchase requests if the department's budget was exhausted. Relying purely on frontend validation was insecure, as it could be bypassed.

How the team addressed it: We implemented server-side validation during the approval routing. Before updating a PR to 'APPROVED', the Flask backend queries the `Departments` table, calculates the remaining budget (`budget_allocated - budget_used`), and verifies it against the `estimated_cost` of the request. If insufficient, the transaction is immediately rejected and a warning is flashed to the user.

Challenge 2: Maintaining Workflow State Consistency

Issue encountered: In our multi-stage pipeline, an order moves from Pending -> Approved -> PO Issued -> Shipped -> Invoiced -> Paid. Initially, advancing the status of a Purchase Order did not update the original Purchase Request, resulting in employees seeing outdated statuses on their dashboards.

How the team addressed it: We engineered a cascading status update mechanism. During advanced operations like `vendor_ship` or `finance_invoice`, the system utilizes JOIN queries to retrieve the `linked_pr_id`. It then executes UPDATE commands on both the downstream tables (Orders/Invoices) and the upstream table (Requests), guaranteeing synchronization across the entire platform.

Proof of Work: Budget Validation Logic

```
    # Retrieve department budget

    dept = conn.execute(

        'SELECT budget_allocated, budget_used FROM Departments WHERE
    dept_id = ?',

        (pr['dept_id'],)

    ).fetchone()


    remaining_budget = dept['budget_allocated'] - dept['budget_used']

    total_cost = pr['estimated_cost'] * pr['quantity']


    if total_cost > remaining_budget:

        flash('Insufficient department budget!', 'danger')

        return redirect(url_for('dashboard'))



    # Proceed with approval and deduct funds

    conn.execute('UPDATE Departments SET budget_used = budget_used + ?
    WHERE dept_id = ?',

                 (total_cost, pr['dept_id']))
```

7. Conclusion

Summary of the project: The Multi-Vendor Procurement system successfully digitizes a complex corporate workflow into an intuitive, secure, and highly responsive web application. It effectively demonstrates the power of relational databases in enforcing business logic, validating financial transactions, and maintaining data integrity across highly segregated user roles. The system is production-ready, featuring full audit logging and secure authentication.

Key learnings: Throughout the development of this platform, the team gained significant hands-on experience in several critical domains:

- **Database Normalization:** Structuring 16 tables to satisfy 3NF, eliminating data redundancy, and isolating financial records.
- **Referential Integrity:** Managing complex foreign key dependencies to ensure orphaned records cannot exist.
- **Complex SQL Operations:** Writing advanced multi-table JOINs and executing sequential transactional updates.
- **Full-Stack Integration:** Connecting a Python Flask backend to a SQLite database and securely rendering that data on a dynamic HTML frontend.
- **Audit and Security:** Implementing comprehensive IP and timestamp logging for every database mutation to ensure strict accountability.

Possible future improvements: While the current system is highly functional, future enhancements could significantly expand its capabilities. We could migrate the backend database from embedded SQLite to a managed PostgreSQL cluster for improved concurrent write performance and scalability. Additionally, integrating an automated email notification system (e.g., via SendGrid) would alert managers immediately when new requests are submitted. Finally, developing external REST API endpoints would allow vendor systems to interface directly with our database, automating the shipping and invoicing phases.